Databases and its Applications

Relational Modelling

Pooja T S
Computer Applications

Databases and its Applications

Relational Modelling

Enhanced ER Models: Specialisation, Generalisation, Aggregation

Pooja T S

Computer Applications

Introduction to Enhanced ER (EER) Model

- The **Enhanced Entity–Relationship (EER) Model** extends the basic ER model with additional semantic concepts.
- It helps represent:
  - Hierarchical relationships among entities.
  - Complex real-world constraints and inheritance.
  - Scenarios where an entity is a subset of another.
- The EER model includes:
  - **Specialisation**
  - **Generalisation**
  - **Aggregation**
- EER is often called the "Semantic Data Model."

Motivation for EER Modelling

- Basic ER models cannot easily express:
  - Hierarchical entity structures.
  - Attribute inheritance between entities.
  - Relationships among relationships.
- EER extensions solve these limitations using abstraction mechanisms:
  - Specialisation → Top-down design.
  - Generalisation → Bottom-up design.
  - Aggregation → Relationship abstraction.
- Goal: Improve the expressiveness and reusability of conceptual models.

Specialisation — Concept

- **Definition:** Specialisation is the process of defining a set of subclasses of an entity type.
- A subclass inherits attributes and relationships of its parent entity.
- Example:
  - `EMPLOYEE` specialised into `FACULTY` and `STAFF`.
  - Each subclass may have additional attributes — e.g., `FACULTY.Subject`.
- Represented as a top-down hierarchy in the EER diagram.

Specialisation — Characteristics

- **Attributes of Specialisation:**
  - **Inheritance:** Subclasses inherit all attributes and relationships.
  - **Additional Attributes:** Unique to subclass.
  - **Subtype Discriminator:** Attribute that determines subclass membership.
- **Example:**

  `EMPLOYEE` (EID, Name, Type)

  If Type = ‘F’, then FACULTY; if Type = ‘S’, then STAFF.
- Note: All subclasses are subsets of the superclass entity.

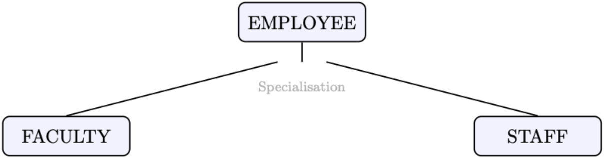


Figure: Specialisation

Specialisation — Constraints

- Two key constraints apply to specialisation:
  - **Disjointness Constraint:**

    Ensures an entity instance belongs to at most one subclass.

    Disjoint (D) vs. Overlapping (O) specialisation.
  - **Completeness Constraint:**

    Specifies whether all superclass entities must appear in subclasses.

    Total (T) → Every entity must belong to a subclass.

    Partial (P) → Some entities may not belong to any subclass.
- These constraints define the structure of the hierarchy precisely.

Generalisation — Concept

- **Definition:** Generalisation is the process of abstracting common attributes from multiple entity types into a higher-level entity.
- It is the reverse of specialisation — a bottom-up approach.
- Example:
  - `CAR`, `BIKE`, and `TRUCK` can be generalised into `VEHICLE`.
  - Common attributes like `RegistrationNo, EngineNo` move to the superclass.
- Useful when entities share a set of common properties and behaviours.

Generalisation — Characteristics

- **Key Properties:**
  - Common attributes moved to a **superclass**.
  - Inherited relationships apply to all subclasses.
  - Avoids redundancy in modelling shared data.
- **Example:**

  $\texttt{CAR(Mileage, RegNo)}, \texttt{TRUCK(Capacity, RegNo)} \Rightarrow \texttt{VEHICLE(RegNo, Type)}$

Generalisation

- Generalisation and Specialisation are symmetric — direction of abstraction differs.

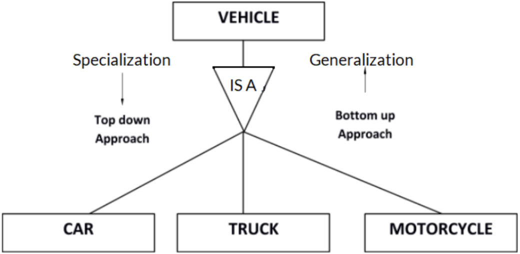
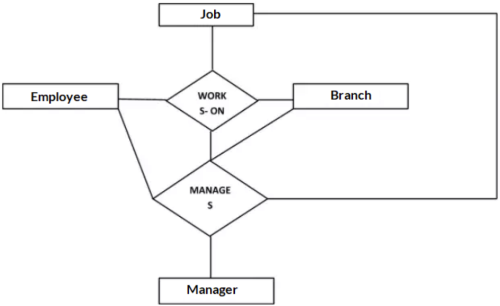


Figure: Class hierarchy can be viewed one of two ways: 1. Specialization (Top Down Approach) 2. Generalization (Bottom Up Approach)

Aggregation — Concept

- **Definition:** Aggregation is an abstraction through which a relationship set is treated as an entity set for further relationships.
- It is used when a relationship itself participates in another relationship.
- Example:
  - `JOB` is assigned to `EMPLOYEE` → relationship `Works_On`.
  - `MANAGER` supervises a `Works_On` relationship → aggregation required.
- Represented as a diamond within a rectangle in EER diagrams.

Aggregation



- Consider a ternary relationship Works On between Employee, Branch and Manager. Now the best way to model this situation is to use aggregation. The relationship set Works On is a higher-level entity set, treated in the same manner as any other entity set. We can create a binary relationship, Manager, between Works On and Manager to represent who manages what tasks.

Aggregation — When to Use

- Use aggregation when:
  - A relationship itself needs to have attributes.
  - A relationship participates in another relationship.
  - The relationship must be treated as a higher-level abstraction.
- **Example Scenario:**

  `EMPLOYEE{Works On JOB` with attribute `Hours`)

  `MANAGER{Monitors{(Works_On)` → Aggregation.
- Tip: Aggregation avoids misrepresentation of "relationship on relationship" semantics.

Comparison — Specialisation vs Generalisation

- Both are abstraction mechanisms but differ in direction and intent:

Feature	Specialisation	Generalisation
Approach	Top-down	Bottom-up
Purpose	Identify sub-entities from a general entity	Combine similar entities into one superclass
Result	Subclasses inherit attributes from parent	Superclass abstracts common attributes
Example	EMPLOYEE → FACULTY, STAFF	CAR, BIKE → VEHICLE

EER Model — Advantages and Use Cases

- **Advantages:**
  - Captures inheritance and hierarchy naturally.
  - Reduces redundancy in data modelling.
  - Enhances semantic clarity of conceptual design.
  - Useful in object-oriented database mapping.
- **Use Cases:**
  - University system with multiple staff types.
  - Banking system with Customer subtypes.
  - Hospital management with Doctor/Patient hierarchies.

Summary — Enhanced ER Concepts

- **Enhanced ER (EER) Model:**
  - Specialisation — divides an entity into sub-entities.
  - Generalisation — combines entities into a higher abstraction.
  - Aggregation — treats relationships as entities.
- EER provides better representation for complex data semantics.
- Next: Mapping EER to Relational Schema (Class 31–32).

Thank You

Pooja T S
Assistant Professor
Department of Computer Applications
poojats@pes.edu
080-26721983 Extn: 233